Relatório – Laboratório 01

Características de Repositórios Populares no GitHub

Disciplina: Laboratório de Experimentação de Software

Data: Março de 2026

Sumário

1. Introdução **3**

2. Metodologia **3**

3. Questões de Pesquisa e Hipóteses Informais **3**

4. Resultados da Análise **5**

Resumo das medianas observadas: 5

4.1 RQ01 — Sistemas populares são maduros/antigos? 5

4.2 RQ02 — Sistemas populares recebem muita contribuição externa? 5

4.3 RQ03 — Sistemas populares lançam releases com frequência? 6

4.4 RQ04 — Sistemas populares são atualizados com frequência? 7

4.5 RQ05 — Sistemas populares são escritos nas linguagens mais populares? 7

4.6 RQ06 — Sistemas possuem alto percentual de issues fechadas? 7

4.7 RQ07 (Bônus) — Linguagem influencia contribuição, releases e atualizações? 9

5. Conclusões e Próximos Passos **11**

6. Referências **12**

1. Introdução

Este relatório apresenta a análise dos **1.000 repositórios com maior número de estrelas no GitHub**, coletados via API GraphQL. O objetivo é investigar características comuns a sistemas populares, como maturidade, frequência de contribuições, lançamento de releases, atualização, linguagem de programação e gestão de issues, relacionando-as com o sucesso medido pela popularidade (estrelas).

As seções a seguir apresentam hipóteses informais para cada questão de pesquisa (RQ), validadas ou refutadas com base nos dados coletados.

2. Metodologia

Os dados foram coletados por meio de um script Python que consome a **API GraphQL do GitHub**. A consulta utiliza paginação via *endCursor* para recuperar os 1.000 repositórios com mais estrelas. A análise foi realizada com base em **valores medianos** para métricas numéricas e **contagem por categoria** para métricas categóricas (como linguagem de programação).

Campo	Descrição
`nameWithOwner`	Nome do repositório (owner/repo)
`createdAt`	Data de criação
`stargazerCount`	Número de estrelas
`pullRequestsMerged`	Total de pull requests aceitas (MERGED)
`releasesTotal`	Total de releases publicadas
`updatedAt`	Data da última atualização
`primaryLanguage`	Linguagem de programação principal
`issuesTotal`	Total de issues (abertas + fechadas)
`issuesClosed`	Total de issues fechadas

3. Questões de Pesquisa e Hipóteses Informais

RQ01 — Sistemas populares são maduros/antigos?

Métrica: Idade do repositório em anos a partir de createdAt.

Hipótese: Espera-se que a maioria dos repositórios populares seja relativamente antiga (mediana acima de 5 anos). Projetos consolidados têm mais tempo para acumular estrelas, construir comunidade e se tornar referências em suas áreas.

RQ02 — Sistemas populares recebem muita contribuição externa?

Métrica: Total de pull requests aceitas (MERGED).

Hipótese: Espera-se que repositórios populares possuam um número elevado de pull requests mescladas. Projetos com muitas estrelas tendem a atrair mais desenvolvedores dispostos a contribuir.

RQ03 — Sistemas populares lançam releases com frequência?

Métrica: Total de releases publicadas.

Hipótese: Espera-se que a maioria dos repositórios ativos tenha um número considerável de releases, mas repositórios do tipo 'awesome lists' podem ter poucas, reduzindo a mediana geral.

RQ04 — Sistemas populares são atualizados com frequência?

Métrica: Tempo em dias desde a última atualização (updatedAt).

Hipótese: Espera-se que os repositórios populares sejam atualizados com frequência (mediana de dias muito baixa), atraindo contribuidores ativos que mantêm o projeto em evolução contínua.

RQ05 — Sistemas populares são escritos nas linguagens mais populares?

Métrica: Linguagem primária (primaryLanguage) de cada repositório.

Hipótese: Espera-se que linguagens amplamente adotadas (JavaScript, Python, TypeScript, Java) dominem entre os repositórios mais populares.

RQ06 — Sistemas populares possuem alto percentual de issues fechadas?

Métrica: Razão entre issuesClosed e issuesTotal.

Hipótese: Espera-se que repositórios populares apresentem uma taxa alta de issues fechadas (mediana > 70%), indicando boa manutenção e atenção às demandas da comunidade.

4. Resultados da Análise

Resumo das medianas observadas:

Questão	Resultado
Mediana de idade (anos)	8,38
Mediana de PRs mescladas	743
Mediana de releases	40
Mediana de dias desde última atualização	6
Mediana do percentual de issues fechadas	88,0%

4.1 RQ01 — Sistemas populares são maduros/antigos?

A mediana de **8,38 anos** confirma a hipótese: a maioria dos projetos no top 1.000 tem existência consolidada. Isso é coerente com a ideia de que tempo e acumulação de comunidade contribuem para o acúmulo de estrelas.

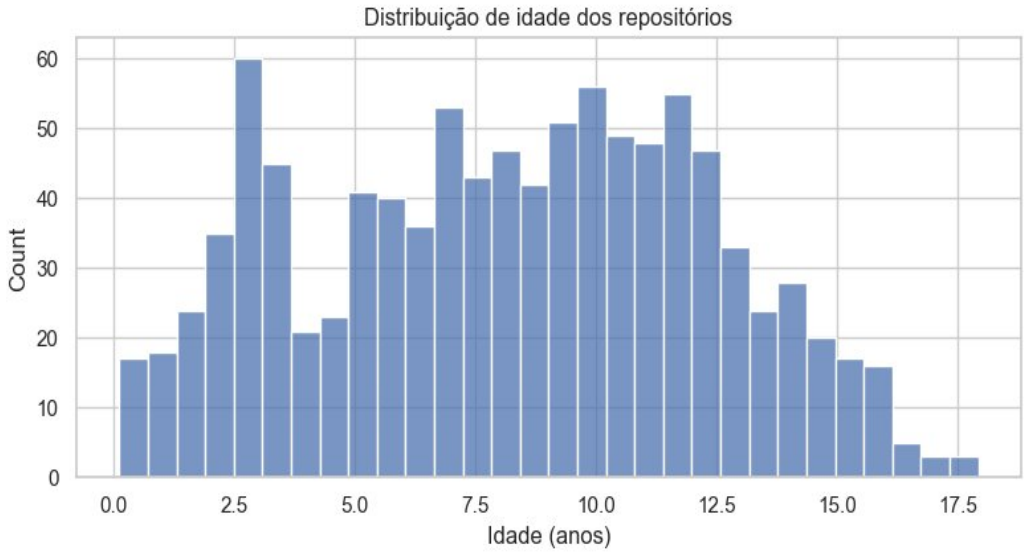

Figura 1: Distribuição de idade dos repositórios (anos).

4.2 RQ02 — Sistemas populares recebem muita contribuição externa?

A mediana alta de **743 PRs mescladas** indica que muitos repositórios recebem volume substancial de contribuições. Observa-se variabilidade grande: projetos do tipo 'listas' têm PRs baixos, enquanto bibliotecas/infraestrutura e grandes frameworks apresentam milhares de PRs.

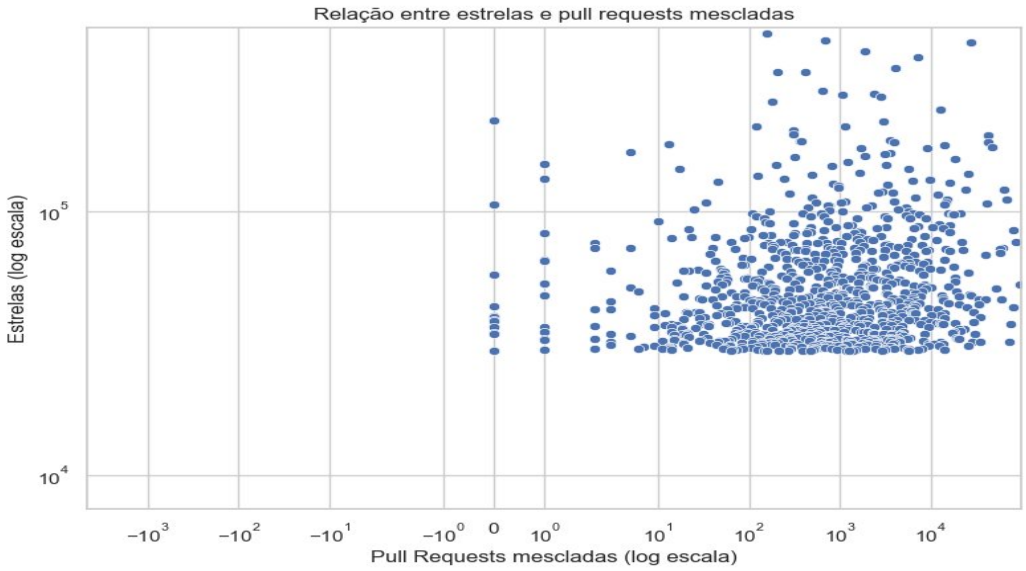

Figura 2: Relação entre número de estrelas e PRs mescladas (escala log).

4.3 RQ03 — Sistemas populares lançam releases com frequência?

A mediana de **40 releases** indica uso razoável de releases no conjunto. A presença de muitos repositórios do tipo documentação/coleção (que podem ter 0 releases) reduz a mediana relativa ao subconjunto de bibliotecas ativas.

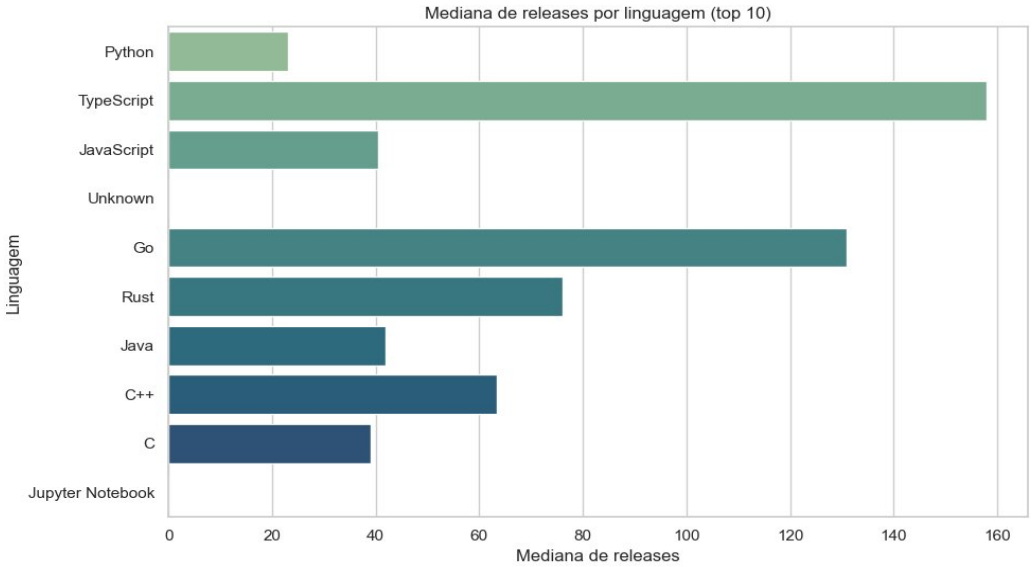

Figura 3: Mediana de releases por linguagem (top 10).

4.4 RQ04 — Sistemas populares são atualizados com frequência?

A mediana de **6 dias** confirma que, no agregado, esses projetos são atualizados frequentemente. Muitos projetos no top 1.000 recebem atualizações contínuas.

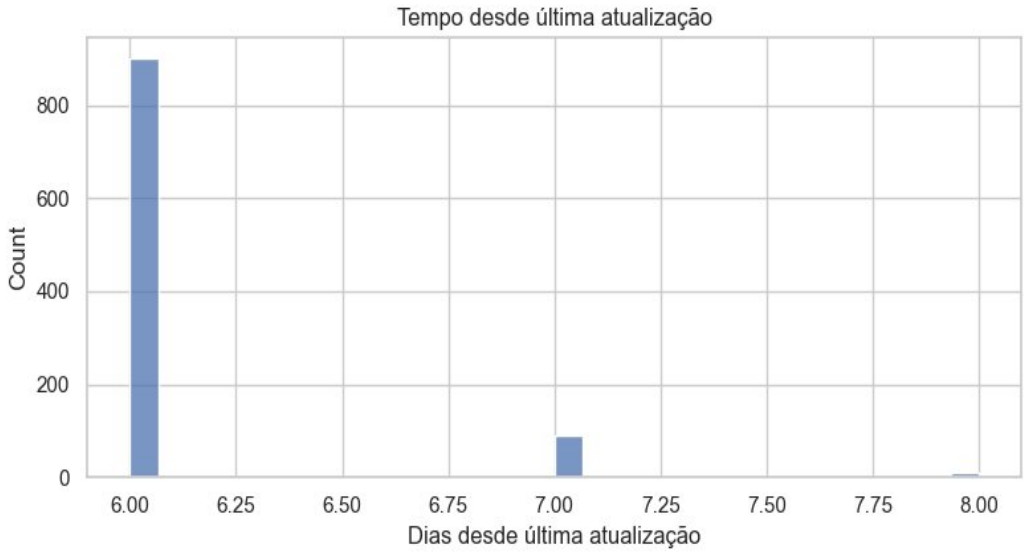

Figura 4: Distribuição do tempo desde a última atualização (dias).

4.5 RQ05 — Sistemas populares são escritos nas linguagens mais populares?

Python (201), TypeScript (160) e JavaScript (114) dominam o ranking, confirmando a hipótese. O grupo 'Unknown' (96) corresponde a repositórios de documentação/listas sem linguagem primária definida.

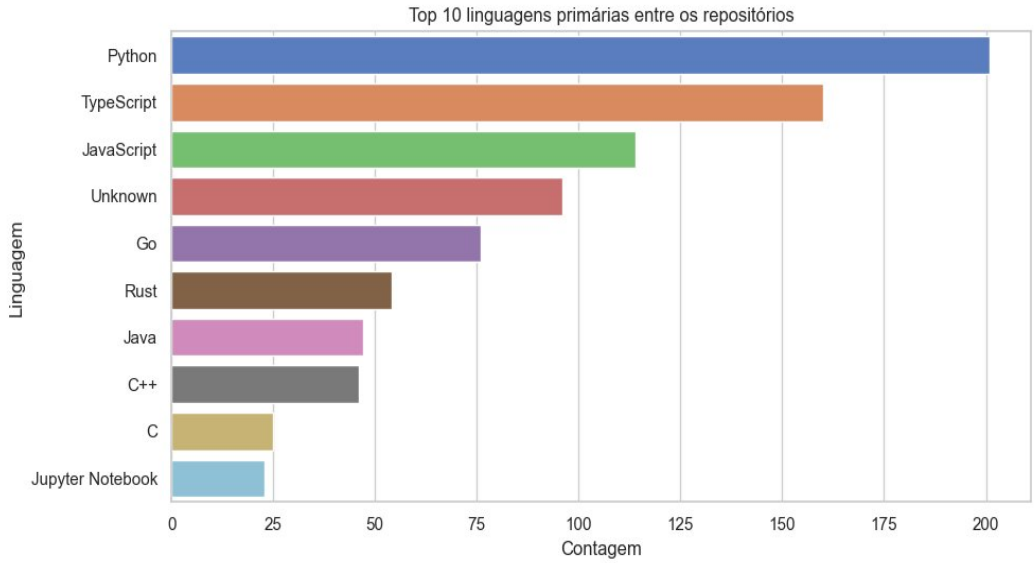

Figura 5: Top 10 linguagens primárias entre os repositórios.

4.6 RQ06 — Sistemas possuem alto percentual de issues fechadas?

A mediana de **88%** sustenta a hipótese: repositórios populares tendem a fechar a maioria das issues, sinalizando manutenção ativa e gestão de backlog.

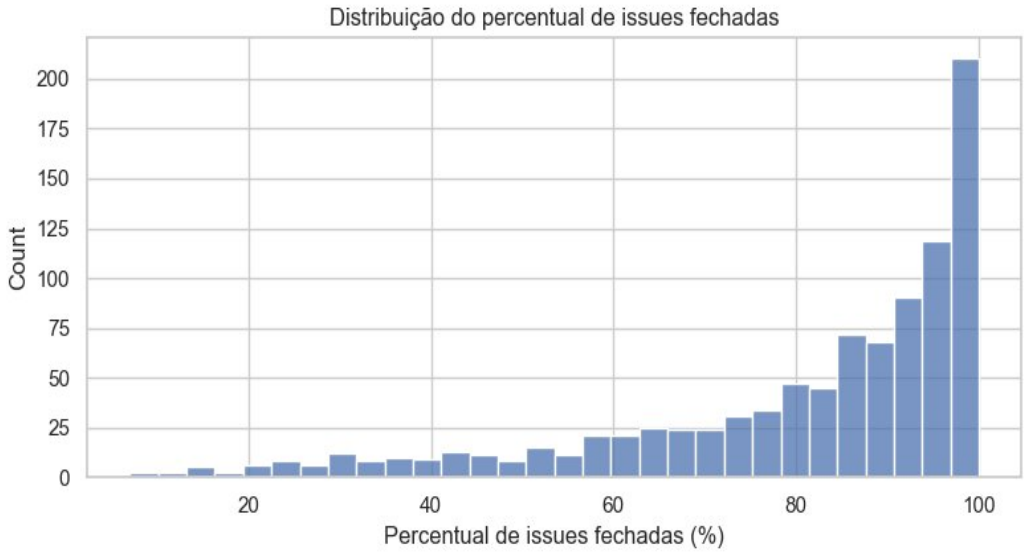

Figura 6: Percentual de issues fechadas (histograma).

4.7 RQ07 (Bônus) — Linguagem influencia contribuição, releases e atualizações?

Foram comparadas as medianas por linguagem com as medianas globais. Os resultados mostram que **TypeScript** e **Rust** apresentam medianas de PRs e releases consideravelmente maiores, indicando ecossistemas com fluxo mais intenso de contribuição e versionamento.

Linguagem	Count	Med. PRs	Med. Releases	Med. Dias	Med. Estrelas
Python	201	621	23	6	42.778
TypeScript	160	2.588	158	6	42.240
JavaScript	114	589	40	6	45.013
Unknown	96	129	0	6	47.529
Go	76	1.508	131	6	39.849
Rust	54	2.371	76	6	41.501
Java	47	605	42	6	37.352
C++	46	982	63	6	39.746
C	25	145	39	6	34.965
Jupyter Notebook	23	88	0	6	45.862

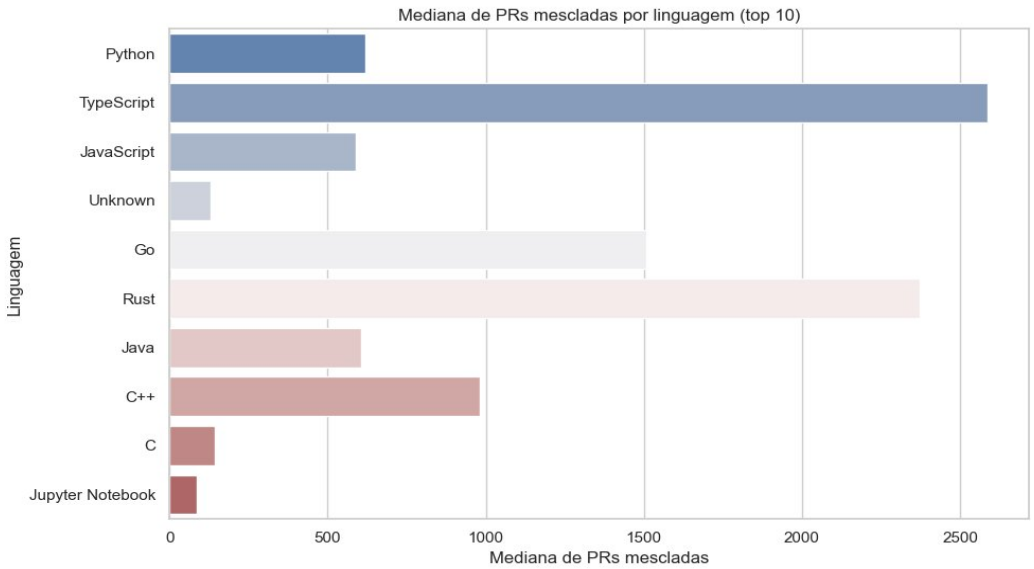

Figura 7: Mediana de PRs mescladas por linguagem (top 10).

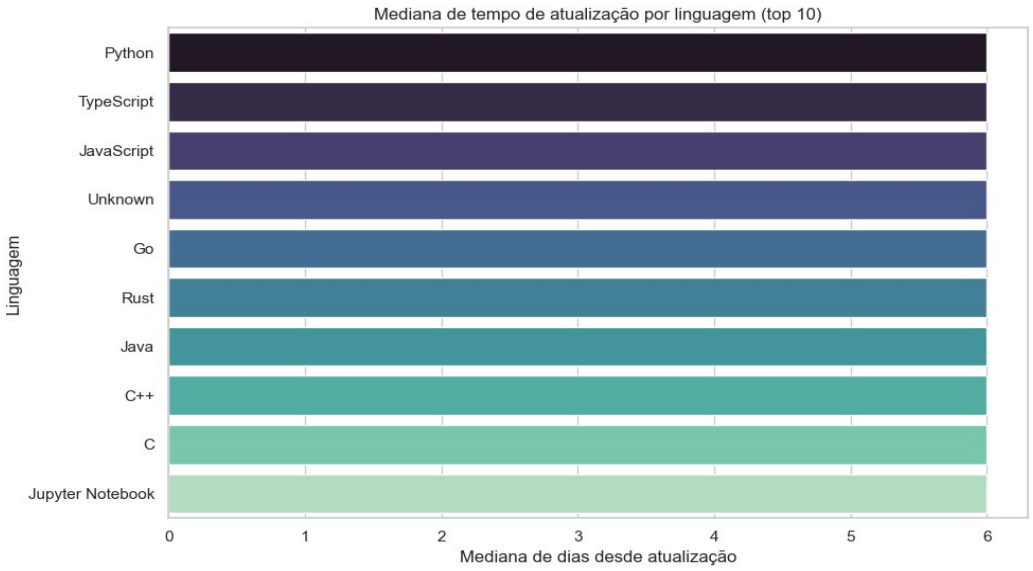

Figura 8: Mediana de dias desde última atualização por linguagem (top 10).

5. Conclusões e Próximos Passos

O conjunto analisado (top 1.000) tende a ser maduro, ativo e com alta taxa de fechamento de issues. As principais conclusões são:

- A **maturidade** é uma característica marcante: mediana de 8,38 anos confirma que popularidade se constrói ao longo do tempo.
- A **atividade de contribuição** é alta: 743 PRs mescladas em mediana indicam comunidades engajadas.
- O **versionamento via releases** é praticado: mediana de 40 releases, com destaque para TypeScript (158) e Go (131).
- A **frequência de atualização** é muito alta: mediana de apenas 6 dias sem atualização.
- A **gestão de issues** é eficiente: 88% das issues são fechadas, indicando manutenção ativa.
- Há **diferenças por linguagem**: TypeScript e Rust destacam-se com maiores medianas de PRs e releases.

Próximos passos recomendados:

- Segmentar por tipo de repositório (biblioteca, framework, lista, tutorial).
- Realizar testes estatísticos para comparar distribuições entre linguagens.
- Criar visualizações adicionais controlando por idade e tipo de projeto.

6. Referências

[1] GitHub, Inc. *GitHub GraphQL API v4*. Disponível em: https://docs.github.com/en/graphql. Acesso em: março de 2026.

[2] Dyer, R., Nguyen, H. A., Rajan, H., & Nguyen, T. N. (2013). *Boa: A Language and Infrastructure for Analyzing Ultra-Large-Scale Software Repositories*. In Proceedings of ICSE 2013.

[3] Borges, H., Hora, A., & Valente, M. T. (2016). *Understanding the Factors That Impact the Popularity of GitHub Repositories*. In Proceedings of ICSME 2016.

[4] Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., & Damian, D. (2014). *The Promises and Perils of Mining GitHub*. In Proceedings of MSR 2014.

[5] Gousios, G. (2013). *The GHTorrent Dataset and Tool Suite*. In Proceedings of MSR 2013.

[6] Python Software Foundation. *Python 3 Documentation*. Disponível em: https://docs.python.org/3/. Acesso em: março de 2026.

[7] ReportLab Inc. *ReportLab PDF Library User Guide*. Disponível em: https://www.reportlab.com/docs/reportlab-userguide.pdf. Acesso em: março de 2026.